AskMyPDF — how it works

Extractive question answering over PDFs · Hugging Face `pipeline("question-answering")` · RoBERTa / XLM-RoBERTa fine-tuned on SQuAD 2.0

1 · Tech flow

what happens from the moment a PDF arrives to the moment an answer appears

1 — PDF arrives `app.py`

Uploaded in the browser or picked from the samples. Saved to `/uploads`.

- bytes handed to the extractor
- cached, so a re-run doesn't re-read it

2 — Text extraction `app/pdf_extractor.py`

pdfplumber first, **PyPDF2** as a fallback.

- `x_tolerance=1.5` — stops words gluing together
- two-column pages read one column at a time
- hyphens, right-to-left scripts and control characters repaired

3 — Is there any text? `looks_scanned`

Under 20 words a page, or half the pages empty, means the pages are **images**.

- yes → carry on to chunking
- no → OCR path (below)

4 — Chunking `app/chunker.py`

The model reads at most **512 tokens**. Long documents are cut into overlapping pieces.

- size **measured** with the model's tokenizer
- overlap keeps boundary answers whole
- each chunk remembers its page numbers

5 — Question answering `app/qa_engine.py`

Every chunk is read with the question: the model marks where the answer **starts** and **ends**.

- runs locally, or on Hugging Face's servers
- also returns a "no answer here" score

6 — Pick & show `app.py`

Chunks ranked by **span score − "no answer" score**, which makes them comparable.

- answer + confidence + page
- source text, answer highlighted
- history, exportable as CSV

→ OCR path — scanned books, and Indian-language PDFs that extract badly

`app/ocr.py` · **pypdfium2** renders each page to an image → **EasyOCR** reads the characters off it. ~20 s per page, so you choose the page range; pages already read are cached. Falls back from GPU to CPU if memory runs out. Recovers Telugu conjuncts the text layer destroys: ఆ → ఆగ్రా.

→ Model choice — made in the sidebar

deepset/roberta-base-squad2 (124M, English) or **deepset/xlm-roberta-large-squad2** (560M, ~100 languages). Both are *extractive*: they copy a span out of your document and never write new text, so they cannot invent a fact — but cannot summarise either.

INPUT PDF	AFTER CHUNKING	QUESTION	MODEL OUTPUT	SHOWN TO THE USER
sample_long.pdf 3 pages · 1,354 words	**4 chunks** 400 words, 50 overlap	"Which computer defeated Garry Kasparov in 1997?"	**Deep Blue** chunk 3 of 4	**76.7%** confidence · pages 2–3

2 · Where each piece lives

one file per job, so each step can be explained on its own

`app.py`
Streamlit screen: upload, question, results, history, settings. No NLP logic.

`app/pdf_extractor.py`
PDF → text. Library fallback, column handling, hyphen and right-to-left repair, scan detection.

`app/ocr.py`
Pages → images → text with EasyOCR. Page ranges, caching, GPU/CPU fallback.

`app/chunker.py`
Text → overlapping chunks. Measures tokens per word; counts characters for Chinese and Japanese.

`app/qa_engine.py`
The Hugging Face pipeline, the "no answer" handling, and choosing between chunks.

`app/hf_inference_api.py`
Optional: same questions answered on Hugging Face's servers instead of locally.

`app/hub_info.py`
Live model details from the Hub API — task, downloads, licence — shown in the sidebar.

`app/config.py`
Every tunable number in one place: chunk sizes, token budget, thresholds, model names.

3 · Development flow

how the project was built, and what each stage changed

STAGE 1 — Foundations

- Checked both models exist on the Hub
- Pinned `transformers<5` — v5 removed the QA pipeline
- Isolated `.venv`; models cached in `/models`

STAGE 2 — Core pipeline

- Extractor → chunker → QA engine, one file each
- Generated sample PDFs with no extra libraries
- Streamlit UI on top, logic kept out of it

STAGE 3 — Testing

- Unit tests for chunk overlap, fallbacks, errors
- End-to-end examples with expected answers
- UI driven headlessly with Streamlit's AppTest

STAGE 4 — Real documents

- Research papers exposed glued words, two-column reading and token overflow
- A scanned Telugu book drove the whole OCR path
- 26 language PDFs built as a test set

STAGE 5 — Hardening & docs

- Memory, GPU and token failures made survivable and legible
- README, user manual, this diagram
- Committed and pushed to GitHub

48 tests · **7/7** English examples · **25/25** Indian-language questions · **45/55** across 26 languages

Python 3.10 · transformers 4.57 · Streamlit · pdfplumber · PyPDF2 · EasyOCR